Object Detection, Image Segmentation

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



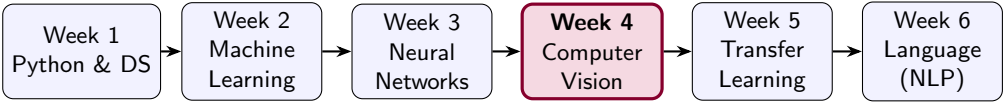
KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 24, Lecture

12 October 2026

Table of Contents

1. Big Picture and Motivation
2. Learning Outcomes
3. Boxes, IoU and How Detectors Are Scored
4. Object Detection: YOLO
5. Image Segmentation: U-Net
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

Where We Are



Today: Day 24 of 42

- **Saturday:** an image is a tensor; convolution, padding, pooling; the three vision tasks by their outputs. **Yesterday:** blocks and stages, LeNet to ResNet, BatchNorm2d, skip connections, global average pooling, data augmentation. Everything so far produced *one label per image*.
- **Today:** the other two tasks. **Detection**: where are the objects (boxes). **Segmentation**: which pixels belong to what (masks). Same convolutional backbone, new heads, and a new way of scoring: overlap.
- **This week:** Day 25 shapes and classification in depth; Day 26 YOLO, U-Net, IoU and mAP with numbers; Day 27 vision habits and augmentation techniques; Friday.

How We Did It Before: A Classifier at Every Window

- **Detection by sliding:** take Saturday's classifier ("person: yes or no"), cut a 64×64 window from the photo, classify it, move the window 8 pixels, repeat; then again with bigger windows for closer people.
- On a 335×344 photo one window size gives $34 \cdot 36 = 1{,}224$ windows; three sizes, about 3,700 classifier passes for one photo. Where the classifier says yes, draw the window.
- **Segmentation by patches:** to label one pixel, classify the patch around it. 115,240 pixels, 115,240 passes.
- The first CNN detectors (2014) still worked this way, running the network on about 2,000 candidate windows per photo: **13 seconds per photo** on a GPU (Girshick et al., 2014).

Photo 335×344	**passes**
one window size, step 8	1,224
three window sizes	about 3,700
one pass per pixel	115,240
2,000 candidate windows (2014)	2,000 (13 s)
YOLO (2016): whole photo at once	1 (0.022 s)

Sources: the window counts are arithmetic on this photo; Girshick et al. (2014), Section 2.2; Redmon et al. (2016), abstract: 45 photos per second.

A video camera delivers 30 photos per second. Until 2016 no detector kept up.

Where It Breaks

Four problems with "classify every window"

- **Cost.** Thousands of passes per photo; 13 seconds is 0.08 photos per second. Most of the work is repeated: neighbouring windows share almost all their pixels, yet each is computed from scratch.
- **Duplicates.** Every window near a person says "yes". One person, twenty boxes. Somebody has to decide which box to keep.
- **Shape.** A square window of a few fixed sizes fits neither a tall pedestrian nor a wide bus. The box should be *predicted*, not chosen from a menu.
- **No context for pixels.** A grey patch is road, a coat or a wall; the patch alone cannot tell. Labelling pixels needs the whole picture *and* pixel precision at once.

The pattern

Run the convolutions **once** over the whole image (Saturday's feature maps already cover every position), then read the maps with a head built for the task: boxes at every grid cell, or a class at every pixel. Score it by **overlap** with the truth.

Today's Idea

In one sentence

A detector and a segmenter are yesterday's CNN with a different **head**: **YOLO** divides the image into a grid and lets every cell predict boxes with a confidence and class scores in one pass, then **non-maximum suppression** removes duplicates; **U-Net** goes down to gain context and back up with **skip connections** to label every pixel. Both are scored by **IoU**, the overlap between prediction and truth.

- **IoU** (intersection over union) says how well a box or a mask matches; **mAP** turns box matches into one score for a detector; **Dice** and mean IoU do it for masks.
- **YOLO** has grown from the 2016 paper into the ultralytics library: one line to train, one to predict, pretrained weights to start from. That is the detector you will use in a competition and in today's lab.

Where you will meet it

Tasks that hand you boxes in a CSV or an XML file, or masks as PNG files, and score by mAP, IoU or Dice. Today's labs train YOLO on Kaggle road signs and a U-Net from scratch on Kaggle MRI scans; Day 26 takes YOLO, U-Net and the metrics apart with numbers.

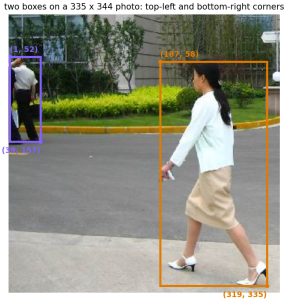
⇒ Learning Outcomes

By the end of this session, you will be able to:

- **Read and write** a bounding box as four numbers, and compute the IoU of two boxes.
- **Explain** precision, recall and average precision for a detector, and what mAP@0.5 means.
- **Explain** how YOLO turns a photo into a grid of box predictions in one pass, why non-maximum suppression is needed, and what changed from the 2016 paper to today's versions.
- **Explain** how U-Net labels every pixel: down for context, up for precision, skip connections for detail; and compute IoU and Dice for a mask.
- **Prepare** a detection dataset in YOLO's format (label files and a data YAML) and run ultralytics' train, val and predict.
- **Train** YOLO on Kaggle road signs and a small U-Net on Kaggle MRI slices, and write their submission files.

Detection: **Boxes, IoU and How Detectors Are Scored**

A Box Is Four Numbers



Source: Penn-Fudan pedestrian database (Wang et al., 2007), image `FudanPed00005`; boxes derived from its masks; `scripts/day24_detection_figures.py`.

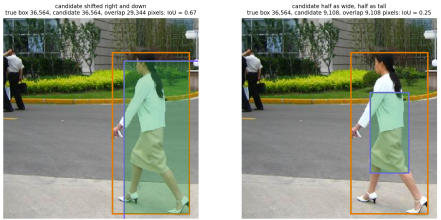
Definition

A **bounding box** is the smallest upright rectangle around an object, written as four numbers in pixels from the image's top-left corner.

Format	The woman's box	Where
(x_1, y_1, x_2, y_2) corners	$(187, 58, 319, 335)$	VOC XML
(x, y, w, h) corner, size	$(187, 58, 132, 277)$	COCO
centre, size, fractions	$(0.76, 0.57, 0.39, 0.81)$	YOLO files

- Same box, three spellings: $w = 319 - 187 = 132$, $h = 335 - 58 = 277$; the centre $(253, 196)$ over the image size $(335, 344)$ gives the fractions.
- A task gives you one spelling per object, plus the class. Reading it wrong is the first bug of a detection notebook.

IoU, With Numbers



Source: the same photo; orange: the true box; purple: a candidate; green: their overlap; `scripts/day24_detection_figures.py`.

$$\mathrm{IoU}(A, B) = \frac{\text{area of } A \cap B}{\text{area of } A + \text{area of } B - \text{area of } A \cap B}$$

- **Shifted 20 right, 15 down:** overlap 29,344, union $36{,}564 + 36{,}564 - 29{,}344 = 43{,}784$, $\mathrm{IoU} = 0.67$. **Half size, inside the true box:** overlap 9,108, union 36,564, $\mathrm{IoU} = 0.25$.
- IoU is 1 for a perfect box, 0 for no overlap; a box counts as **correct at IoU** ≥ 0.5 with the right class. The first candidate is a hit, the second a miss although it sits on the person.

Precision and Recall for Boxes

Definition

Match each predicted box to the true box of the same class it overlaps most. **True positive**: IoU ≥ 0.5 and that true box was not already taken. **False positive**: otherwise. **False negative**: a true box nobody matched.

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}$$

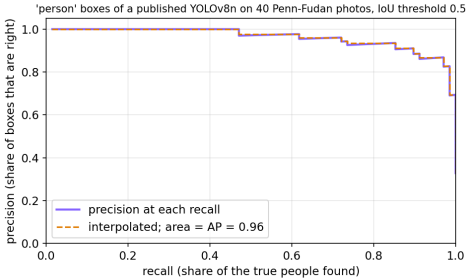
Precision: the share of your boxes that are right. Recall: the share of true objects found.

With numbers

Ten people in a set of photos; the detector draws 12 boxes, 9 of them hits: $TP = 9$, $FP = 3$, $FN = 1$. Precision $9/12 = 0.75$, recall $9/10 = 0.90$. Lower the score threshold and it draws 20 boxes with 10 hits: precision 0.50, recall 1.00.

- Each box has a **score**; your threshold trades precision for recall, so the next metric sweeps all thresholds.

Average Precision and mAP



Source: ultralytics' YOLOv8n with its published COCO weights, run on the first 40 Penn-Fudan photos (68 people); boxes matched at IoU 0.5; `scripts/day24_detection_figures.py`.

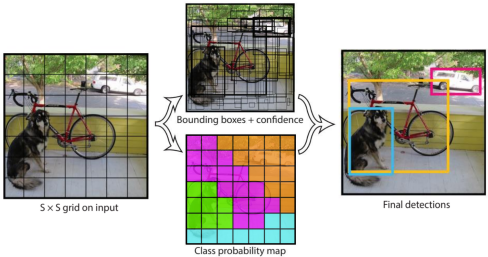
Definition

Sort one class's predicted boxes by score. Walk down the list, computing precision and recall after each box; the area under precision against recall is the **average precision** (AP) of that class. **mAP** is the mean of AP over the classes.

- Here: 207 boxes above score 0.05, 68 true people, $\mathrm{AP} = 0.96$: precision stays near 1 until almost everyone is found, then falls.
- **mAP@0.5** (PASCAL VOC) matches at IoU 0.5; **mAP@[.5:.95]** (COCO) averages the thresholds 0.5 to 0.95 and rewards tight boxes. The numbers differ a lot.

Detection: **YOLO**

YOLO's Idea: One Look

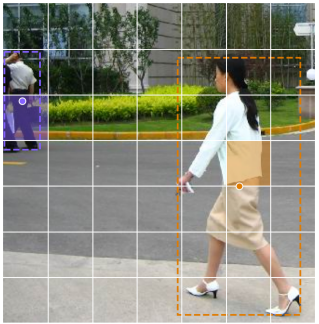


Source: Redmon, Divvala, Girshick and Farhadi (2016), Fig. 2.

- **You Only Look Once:** the whole photo goes through the CNN once. The last feature map is read as an $S \times S$ **grid**; every cell predicts B boxes with a confidence each, and C class probabilities. "A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation" (Redmon et al., 2016).
- Low-confidence boxes are dropped, duplicates removed by non-maximum suppression: the right-hand picture, 45 photos per second on a 2016 GPU.

The Grid, On Our Photo

a 7 x 7 grid: the cell holding an object's centre is responsible for its box



Source: Penn-Fudan image FudanPed00005 with a 7×7 grid; dots mark the box centres; `scripts/day24_detection_figures.py`.

- $S = 7$ on a 335×344 photo: cells of 47.9×49.1 pixels. The woman's centre is $(253, 196)$: column $253/47.9 = 5.3$, row $196/49.1 = 3.99$, so **cell (row 3, column 5)** owns her.
- That cell predicts the centre's offset inside the cell, $(0.29, 1.00)$; the box size as a fraction of the image, $w = 0.39$, $h = 0.81$; a **confidence** trained to equal the IoU of its box with the truth; and the class scores.
- The man's centre $(20, 104)$ lands in cell (row 2, column 0): offset $(0.42, 0.13)$, size $(0.11, 0.31)$. Every other cell learns confidence 0.
- The "classifier at every window" became *one* network whose output has a slot for every cell.

The Output Tensor and the Loss, With Numbers

The output

$S \times S \times (5B + C)$ numbers. PASCAL VOC: $S = 7$, $B = 2$, $C = 20$, so $7 \times 7 \times 30$, that is 1,470 numbers for the whole photo, in one forward pass.

Per cell	Count	Meaning
box 1: x, y, w, h, confidence	5	where, size, how sure
box 2: the same	5	a second guess
class probabilities	20	which object
total	30	one cell's slot

The loss, in words

Squared errors, added up over the grid:

- **box error** on $x, y, \sqrt{w}, \sqrt{h}$ in the responsible cell, weighted $\times 5$;
- **confidence error**: towards the IoU at an object, towards 0 elsewhere ($\times 0.5$: most cells are empty);
- **class error** in the responsible cells.

The square roots make a 10-pixel error count more on a small box.

Non-Maximum Suppression, Measured



Source: ultralytics' YOLOv8n (published COCO weights) on Penn-Fudan image `FudanPed00005`, person boxes with score above 0.3, with suppression switched off and on; `scripts/day24_detection_figures.py`.

Definition

Non-maximum suppression: sort the boxes of a class by score; keep the best; delete every remaining box whose IoU with it is above a threshold (0.5 to 0.7); repeat with the next survivor.

- 19 boxes became 2, one per person, scores 0.87 and 0.84: the rule kept the best of each pile. Ultralytics runs it for you (IoU 0.7); Day 26 runs it by hand.

From the 2016 Paper to Today's YOLO

Version	Year	What it added
YOLOv1	2016	the grid, $7 \times 7 \times 30$, one pass: 63.4% mAP on VOC 2007 at 45 photos per second
YOLOv2	2017	BatchNorm, anchor boxes (preset box shapes per cell), higher resolution
YOLOv3	2018	boxes predicted at three map scales, so small objects too; the 80 COCO classes
YOLOv5	2020	the ultralytics PyTorch library: a data YAML, `train`, `val`, `predict`
YOLOv8	2023	anchor-free head; segmentation and pose models in the same library
YOLO11	2024	the current ultralytics family, sizes n to x

YOLOv8 size	n	s	m	l	x
parameters (millions)	3.2	11.2	25.9	43.7	68.2
COCO mAP@[.5:.95]	37.3	44.9	50.2	52.9	53.9

Sources: Redmon et al. (2016); Redmon and Farhadi (2017, 2018); the ultralytics documentation's YOLOv8 model table (COCO val, 640 pixels).

- The idea never changed: one pass, a grid of predictions, NMS. What grew is the number of scales, how boxes are parameterised, and the tooling. Pick the size by your clock: `n` trains in minutes on a free GPU; `x` is for a competition's last day.

In Code: YOLO's Files and the Three Calls

YOLO's label format

One text file per image, one line per box: `class cx cy w h`, the centre and the size as **fractions** of the image, classes numbered from 0. A `data.yaml` names the image folders and the classes.

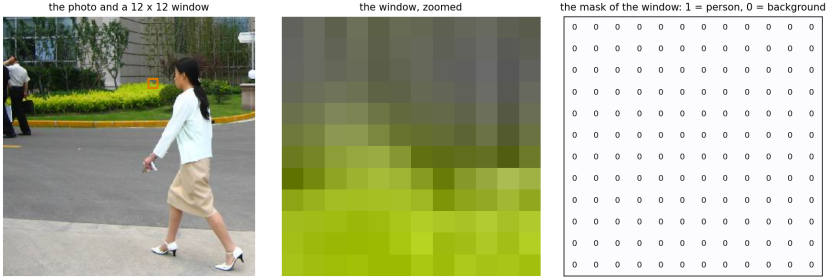
The woman's box $(187, 58, 319, 335)$ in a 335×344 photo: $c_x = 253/335 = 0.755$, $c_y = 196.5/344 = 0.571$, $w = 132/335 = 0.394$, $h = 277/344 = 0.805$, so her line in `labels/FudanPed00005.txt` is `0 0.755 0.571 0.394 0.805`.

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")                                  # start from the COCO weights
model.train(data="signs.yaml", epochs=15, imgsz=416)        # the loop, loss and augmentation
    live here
m = model.val()                                             # scores the YAML's val split
print(m.box.map50, m.box.ap50)                              # mAP@0.5, and AP per class
r = model.predict("road7.png", conf=0.5)[0]
print(r.boxes.xyxy, r.boxes.conf, r.boxes.cls)              # boxes after NMS, in pixels
```

- **train** writes `runs/detect/train/`: `weights/best.pt` and `results.csv` with the loss and mAP of every epoch. The files you prepare and the metric you read are yours, and that is where things go wrong.

Segmentation: **U-Net**

A Mask Is a Grid of Class Numbers



Source: Penn-Fudan image `FudanPed00005` and its human-drawn mask; `scripts/day24_detection_figures.py`.

- The label of a segmentation task is an image of the **same height and width** whose pixels are class numbers: 0 background, 1 person; with k classes, 0 to $k-1$. It is stored as a PNG and loaded like a photo, without colour conversion.
- This photo: 14,738 of its 115,240 pixels are person, 12.8%. The window shows the boundary of a shoulder: the mask is as sharp as the person who drew it.

Classification at Every Pixel, and the Problem

Definition

A segmenter outputs k scores **per pixel**: a tensor (k, H, W). The mask is the argmax over the k channels at each pixel; the loss is cross-entropy averaged over all $H \times W$ pixels. With two classes, one channel and `BCEWithLogitsLoss`.

- Saturday's network cannot do it as it is: three poolings turn 344×335 into 43×41 and `Flatten` throws the positions away.
- Drop the pools and the map stays 344×335, but every unit sees only a few pixels (yesterday's receptive field): no context, the "grey patch" problem of the motivation.

The way out

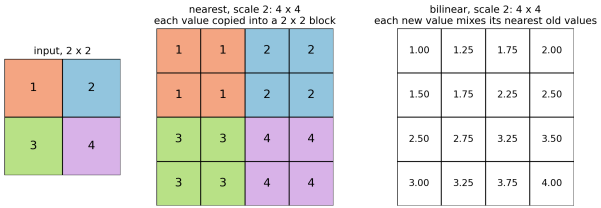
Go **down** with blocks and pools, as yesterday, to gain context; then go **back up** to full size to decide pixel by pixel. The next two frames are the two halves.

	classification	**segmentation**
output	(k)	(k, H, W)
loss	cross-entropy	cross-entropy per pixel
last layer	`Linear`	`Conv2d(., k, 1)`
metric	accuracy	IoU, Dice

Going Back Up: Nearest and Bilinear

Definition

Upsampling makes a feature map larger. The encoder shrank 344×335 to 43×41; the mask needs a score at every pixel, so the decoder has to go back up.



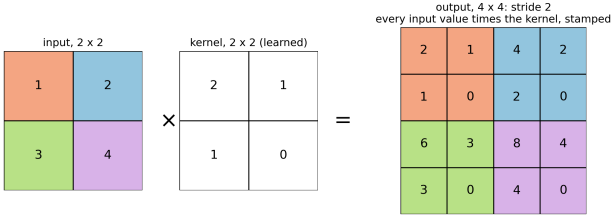
Source: values computed with `F.interpolate`; `scripts/day24_upsampling_figures.py`.

- **Nearest:** every value is copied into a 2×2 block. **Bilinear:** every new value is a weighted mix of its old neighbours: $1, 1.25, 1.75, 2$.
- Both are fixed rules with **zero parameters** (`F.interpolate`). They cannot learn what an edge looks like at the larger size; the decoder wants weights it can train.

Transposed Convolution: Every Value Paints a Patch

Definition

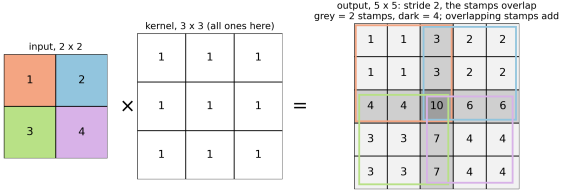
A **transposed convolution** has a $k \times k$ kernel of learned weights and a stride s. Each input value multiplies the whole kernel and the result is **stamped** onto the output, s cells apart; overlapping stamps add.



Source: values computed with `F.conv_transpose2d`; `scripts/day24_upsampling_figures.py`.

With numbers: the input value 3 (bottom left) times the kernel $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ gives $\left(\begin{smallmatrix} 6 & 3 \\ 3 & 0 \end{smallmatrix}\right)$, stamped at rows 2 to 3, columns 0 to 1. With $k = 2$ and $s = 2$ the four stamps tile the output: 2×2 becomes 4×4. A kernel of all ones is exactly nearest upsampling; training moves those four numbers.

Transposed Convolution: Overlap, Size and Code



Source: values computed with `F.conv_transpose2d`; `scripts/day24_upsampling_figures.py`.

With $k = 3$ and $s = 2$ the stamps overlap and add: the centre cell gets $1 + 2 + 3 + 4 = 10$. So a transposed convolution can blend neighbours, not only copy them.

Output size

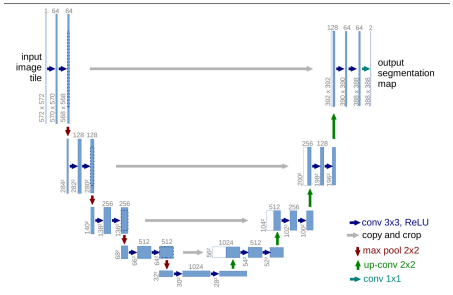
$$H_{\text{out}} = (H_{\text{in}} - 1) \cdot s + k$$

$k = 2, s = 2$: $2H_{\text{in}}$, undoing a 2×2 pool (the U-Net choice). $k = 3, s = 2$: $2H_{\text{in}} + 1$, with overlaps.

```
up = nn.ConvTranspose2d(128, 64,
                        kernel_size=2,
                            stride=2)
x = torch.zeros(1, 128, 32, 32)
print(up(x).shape)      # (1, 64, 64,
    64)
```

Parameters like a convolution: $C_{\text{in}} \cdot C_{\text{out}} \cdot k \cdot k + C_{\text{out}} = 32{,}832$ here. Papers also say "deconvolution"; nothing is inverted, the convolution's arithmetic just runs the other way.

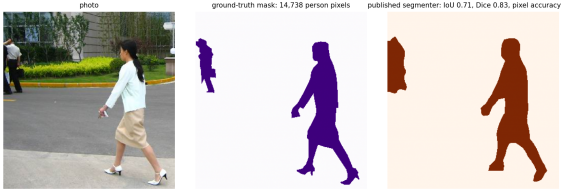
U-Net: Down for Context, Up for Precision, Skips for Detail



Source: Ronneberger, Fischer and Brox (2015), Fig. 1.

- **Left, down:** yesterday's blocks and 2×2 pools, channels doubling 64 to 1024: "a contracting path to capture context".
- **Right, up:** transposed convolutions halve the channels and double the map, then two convs: "a symmetric expanding path that enables precise localization".
- **Grey arrows, the skips:** the encoder map of the same size is **concatenated** onto the decoder map, so the exact position of every edge comes back; stacked as channels, not added as in ResNet.
- **End:** a 1×1 conv to k channels. Trained on 30 microscope images with heavy augmentation; under a second per 512×512 image.

Scoring a Mask: IoU and Dice, With Numbers



Source: torchvision's DeepLabV3 MobileNetV3 (published weights) on Penn-Fudan image `FudanPed00005`, its "person" channel against the human-drawn mask; `scripts/day24_detection_figures.py`.

$$\mathrm{IoU} = \frac{TP}{TP + FP + FN}, \qquad \mathrm{Dice} = \frac{2\,TP}{2\,TP + FP + FN}$$

TP: pixels of the class predicted right; *FP*: background pixels called the class; *FN*: pixels of the class that were missed.

- $TP = 13{,}157$, $FP = 3{,}808$, $FN = 1{,}581$: $\mathrm{IoU} = 13{,}157/18{,}546 = 0.71$, $\mathrm{Dice} = 26{,}314/31{,}703 = 0.83$. Dice is never below IoU; both ignore the background.
- **Pixel accuracy** is 0.953, but all-background scores 0.872: with 87% background, accuracy barely moves. Report IoU or Dice per class, and mean IoU over classes.

In Code: A Small U-Net Class

```
class UNetSmall(nn.Module):           # block(c_in, c_out): conv, BatchNorm, ReLU, twice
    def __init__(self, n_classes=1):
        super().__init__()
        self.down1, self.down2, self.bottom = block(3, 32), block(32, 64), block(64, 128)
        self.up2, self.dec2 = nn.ConvTranspose2d(128, 64, 2, stride=2), block(128, 64)
        self.up1, self.dec1 = nn.ConvTranspose2d(64, 32, 2, stride=2), block(64, 32)
        self.out = nn.Conv2d(32, n_classes, kernel_size=1)
    def forward(self, x):
        d1 = self.down1(x)                              # (32, H, W)
        d2 = self.down2(F.max_pool2d(d1, 2))            # (64, H/2, W/2)
        b = self.bottom(F.max_pool2d(d2, 2))            # (128, H/4, W/4)
        u2 = self.dec2(torch.cat([self.up2(b), d2], dim=1))    # up, stack the skip: 64 +
            64
        u1 = self.dec1(torch.cat([self.up1(u2), d1], dim=1))   # 32 + 32
        return self.out(u1)                             # (n_classes, H, W) logits
loss_fn = nn.BCEWithLogitsLoss()      # two classes: one channel, the mask as floats 0 / 1
```

- Two levels instead of U-Net's four, the same three ideas: down with pools, up with transposed convolutions, `torch.cat` for the skips. The loop is Day 16's; the metric is Dice on `(logits > 0)`.

Beyond Masks per Class: Instance and Panoptic, by Name

Task	Output	Model family	Metric
semantic	one class per pixel	FCN (Long et al., 2015), U-Net, DeepLab	mean IoU, Dice
instance	a mask per object	YOLOv8-seg; Mask R-CNN (He et al., 2017)	mask mAP
panoptic	objects counted, background labelled	Panoptic FPN	PQ

- Saturday's segmentation panel coloured the two people differently: that was an **instance** mask. YOLOv8-seg gets it by predicting a small mask inside every box it detects, so detection and segmentation meet in one network.
- Competitions at this level ask for semantic masks or boxes; instance and panoptic tasks appear by name so you recognise them. Day 26 stays with YOLO and U-Net.

Detection: **Olympiad Practice**

Practice Task: Road Signs

Task

Kaggle's *Road Sign Detection* set: 877 street photos of about 300×400 pixels; for each photo an XML file lists every sign with its class, `trafficlight`, `stop`, `speedlimit` or `crosswalk`, and its box as `xmin, ymin, xmax, ymax`. A hidden set of photos is scored by mAP at IoU 0.5. Hand in one row per predicted box: photo, class, the four corners, a score.

- **Question 1:** which of the three tasks is this, and why does one photo give a variable number of rows?
- **Question 2:** YOLO wants its own label files. Write the label line for a traffic light with box $(98, 62, 208, 232)$ in a 267×400 photo (width 267, height 400), class index 0.
- **Question 3:** your box for that light comes out as $(100, 60, 210, 230)$. Does it count as a hit at IoU 0.5?

Source: Kaggle dataset `andrewmvd/road-sign-detection` (877 photos with PASCAL VOC style XML annotations); class names and counts are the dataset's own.

Solution Idea and Code

Recognise: detection, a list of labelled boxes per photo, scored by mAP. The work is the data format; YOLO's train, val and predict do the rest.

```
# signs.yaml                                    # one label file per photo, in labels/train
    and labels/val
# path: datasets/signs                          # 0 0.573 0.367 0.412 0.425   <- class cx
    cy w h
# train: images/train
# val: images/val
# names: [trafficlight, stop, speedlimit, crosswalk]
model = YOLO("yolov8n.pt")
model.train(data="signs.yaml", epochs=15, imgsz=416)
print(model.val().box.map50)                    # mAP@0.5 on the val split
```

1. Centre $x = (98 + 208)/2 = 153$, $153/267 = 0.573$; centre $y = (62 + 232)/2 = 147$, $147/400 = 0.367$; $w = 110/267 = 0.412$; $h = 170/400 = 0.425$. Line: `0 0.573 0.367 0.412 0.425`.
2. Overlap in x: from $\max(100, 98) = 100$ to $\min(210, 208) = 208$, 108 pixels; in y: from 62 to 230, 168 pixels. Overlap $108 \cdot 168 = 18{,}144$; each box $110 \cdot 170 = 18{,}700$; union 19,256; $\mathrm{IoU} = 0.94$: a hit.
3. **Check:** draw ten label files back onto their photos before training (a swapped x and y shows at once); `results.csv` must show mAP@0.5 rising; the submission uses only the four class names.

Detection & Segmentation: **Limitations & What Comes Next**

Limitations

- **Nobody trains a detector from scratch on a small set.** Every YOLO run in a competition starts from the published COCO weights; today's lab does too. Why that works is Week 5's subject.
- **Labels are expensive.** A box takes seconds to draw, a mask minutes. Small sets with few boxes per class give noisy mAP, and the public and hidden scores can differ by several points.
- **Small, crowded and overlapping objects.** A grid cell owns one object; two people in one cell, or a sign of 10 pixels, are where YOLO-style detectors lose points. Non-maximum suppression also deletes a true neighbour whose box overlaps too much.
- **Pixel accuracy lies.** With 90% background pixels, predicting "background everywhere" scores 0.90. Report IoU or Dice per class, never plain accuracy.

The common mistake

Box formats. `(x1, y1, x2, y2)` against `(x, y, w, h)`, pixels against fractions of the image, and boxes left unchanged after the image was resized. Draw ten training boxes on their photos before training; a wrong format shows at once.

What Comes Next

- **Day 25, self-study:** kernels, padding and shape debugging in depth, then one image-classification run end to end.
- **Day 26, self-study:** YOLO's output tensor and loss, U-Net's blocks, and IoU, mAP and Dice computed by hand on small examples.
- **Day 27, Olympiad practice:** vision habits that win points, augmentation that moves boxes and masks along with the photo, test-time augmentation.
- **Friday:** the Week 4 competition.
- **Week 5:** the pretrained backbones used today, explained: transfer learning, fine-tuning, embeddings.

Detection & Segmentation: **Summary**

Summary

- **A box** is four numbers in one of three spellings; **IoU** is overlap over union, 0.67 for a box shifted by 20 pixels, 0.25 for one half the size; a box counts at IoU ≥ 0.5.
- **Precision, recall, AP, mAP:** sort the boxes by score, sweep the threshold, take the area under precision against recall, average over classes. mAP@0.5 is the VOC rule, mAP@[.5:.95] the COCO rule.
- **YOLO** reads the last feature map as an $S \times S$ grid; each cell predicts B boxes with confidences and C class scores, $7 \times 7 \times 30$ numbers in one pass; **NMS** removes duplicates by IoU (19 boxes to 2 on our photo). 63.4% mAP at 45 photos per second in 2016; today the ultralytics versions, sizes n to x.
- **A mask** is a grid of class numbers; a segmenter outputs (k, H, W) scores and a per-pixel cross-entropy loss.
- **U-Net** goes down for context, up with transposed convolutions for precision, and concatenates skip connections for detail; **IoU** and **Dice** score the mask (0.71 and 0.83 for the published segmenter), pixel accuracy does not.
- **In code:** YOLO wants label files of `class cx cy w h` fractions and a data YAML, then `train`, `val`, `predict`; a small U-Net is yesterday's blocks, two transposed convolutions and two `torch.cat`, trained with Week 3's loop.

Detection & Segmentation: **References & Resources**

References & Resources

[1] Girshick, R., Donahue, J., Darrell, T. and Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *CVPR*. The 2,000-window detector of the motivation and its 13 seconds per image (Section 2.2).

[2] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016). You only look once: unified, real-time object detection. *CVPR*. Fig. 2 reused; 45 frames per second, 63.4% mAP on VOC 2007; the grid, the output tensor and the loss.

[3] Redmon, J. and Farhadi, A. (2017). YOLO9000: better, faster, stronger. *CVPR*. Redmon, J. and Farhadi, A. (2018). YOLOv3: an incremental improvement. arXiv:1804.02767. Anchors, BatchNorm, three scales.

[4] Ultralytics YOLO documentation (YOLOv5, YOLOv8, YOLO11): the data YAML and label format, `train`, `val`, `predict`, and the YOLOv8 model table (parameters, COCO mAP). https://docs.ultralytics.com

[5] Long, J., Shelhamer, E. and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *CVPR*. Classification at every pixel with a convolutional head.

[6] Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI*. Fig. 1 reused; contracting and expanding paths, concatenation skips, training from 30 images with augmentation.

[7] He, K., Gkioxari, G., Dollár, P. and Girshick, R. (2017). Mask R-CNN. *ICCV*. Instance segmentation as Faster R-CNN plus a mask head.

References & Resources (cont.)

[8] Everingham, M. et al. (2010). The PASCAL visual object classes (VOC) challenge. *IJCV* 88. The IoU 0.5 matching rule and average precision. Lin, T.-Y. et al. (2014). Microsoft COCO: common objects in context. *ECCV*. mAP averaged over IoU thresholds 0.5 to 0.95.

[9] Wang, L., Shi, J., Song, G. and Shen, I. (2007). Object detection combining recognition and segmentation. *ACCV*. The Penn-Fudan pedestrian photos and masks behind every measured figure of this deck. https://www.cis.upenn.edu/~jshi/ped_html/

[10] torchvision documentation: `models.segmentation` (DeepLabV3, its published weights used once for the mask figure), `nn.ConvTranspose2d`, `ops.nms`, `ops.box_iou`. https://pytorch.org/vision/stable/models.html

[11] Lab datasets from Kaggle through `kagglehub`: *Road Sign Detection* (`andrewmvd/road-sign-detection`, 877 photos with VOC XML boxes) for Lab 1 and the practice task; *Brain Tumor Segmentation* (`nikhilroxtomar/brain-tumor-segmentation`, the 3,064 MRI slices and masks of Cheng et al., 2015) for Lab 2.

[12] Figures without another source: `slides-builder/scripts/day24_detection_figures.py` and `day24_upsampling_figures.py` (upsampling and transposed-convolution grids, values from `F.interpolate` and `F.conv_transpose2d`). No model was trained for this deck; the only model runs are published weights (YOLOv8n, DeepLabV3) applied once to public photos, and every comparison number is quoted from the sources above.